

Complete Data Cleaning Process

Hands-on with the Titanic Dataset

Machine Learning Practical Series

1 Learning Objectives

After completing this material, you will be able to:

- Load datasets using `pandas` and `seaborn`.
- Perform initial data inspection and understand dataset structure.
- Identify and handle missing values using appropriate strategies.
- Detect and remove duplicate records.
- Standardize text data and convert data types.
- Detect and treat outliers using the IQR method.
- Prepare a clean dataset for Machine Learning.

2 Importing Libraries and Loading the Dataset

We begin by importing the required Python libraries and loading the Titanic dataset, which is available directly through Seaborn.

```
1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 # Load the Titanic dataset
7 df = sns.load_dataset("titanic")
8
9 # Display the first 5 records
10 df.head()
```

Quick check:

- How many rows are present?
- How many columns are present?
- Which column appears to be the target variable?

3 Understanding the Dataset Structure

3.1 Shape of the Dataset

```
1 df.shape
```

Expected output: (891, 15) — meaning **891 passengers** and **15 features**.

3.2 Column Names

```
1 df.columns
```

3.3 Data Types

```
1 df.dtypes
```

3.4 Complete Dataset Information

```
1 df.info()
```

Observe:

- Which columns are numeric?
- Which columns are categorical?
- Which columns contain missing values?

4 Descriptive Statistics

4.1 Numeric Columns

```
1 df.describe()
```

4.2 Categorical Columns

```
1 df.describe(include="object")
```

Questions to explore:

1. What is the average passenger age?
2. What is the maximum fare paid?
3. Which is the most common embarkation town?

5 Missing Value Analysis

5.1 Count of Missing Values

```
1 df.isnull().sum()
```

5.2 Percentage of Missing Values

```
1 missing_percent = (df.isnull().sum() / len(df)) * 100
2 missing_percent.sort_values(ascending=False)
```

5.3 Visualizing Missing Values

```
1 plt.figure(figsize=(10, 5))
2 sns.heatmap(df.isnull(), cbar=False)
3 plt.title("Missing Values Heatmap")
4 plt.show()
```

Think about it: Why is the `deck` column mostly missing? Would you keep it or remove it?

6 Handling Missing Values

6.1 General Strategy

Missing %	Recommended Action
< 5%	Fill with mean / median / mode
5% – 30%	Investigate and decide carefully
> 50%	Consider dropping the column

Data Type	Use	Why?
Numerical & Normally Distributed	Mean	The mean uses all observations in the dataset and provides the most accurate measure of central tendency when the data is approximately symmetric and free from extreme outliers.
Numerical & Skewed / Outliers Present	Median	The median is resistant to extreme values and outliers because it depends only on the middle position of the ordered data, making it a better representation of the center for skewed distributions.
Categorical / Text Data	Mode	The mode identifies the most frequently occurring category or value. Since categorical data cannot be averaged, the mode is the appropriate measure for imputing missing values.

Table 1: Guidelines for Choosing Mean, Median, and Mode for Missing Value Imputation

6.2 Filling the age Column

Check distribution first:

```
1 df["age"].hist()
2 plt.title("Age Distribution")
3 plt.show()
```

Since age is skewed, we fill with the **median**:

```
1 df["age"] = df["age"].fillna(df["age"].median())
```

6.3 Filling the embarked Column

```
1 df["embarked"] =  
  df["embarked"].fillna(df["embarked"].mode()[0])
```

6.4 Filling the embark_town Column

```
1 df["embark_town"] =  
  df["embark_town"].fillna(df["embark_town"].mode()[0])
```

6.5 Dropping the deck Column

Check missing percentage:

```
1 df["deck"].isnull().mean() * 100
```

Since more than 70% of values are missing, drop the column:

```
1 df.drop(columns=["deck"], inplace=True)
```

6.6 Verify

```
1 df.isnull().sum()
```

7 Handling Duplicate Records

7.1 Detect Duplicates

```
1 df.duplicated().sum()
```

7.2 Remove Duplicates

```
1 df = df.drop_duplicates()
```

7.3 Verify

```
1 df.duplicated().sum()
```

8 Data Type Cleaning

Check current data types:

```
1 df.dtypes
```

Convert boolean columns to integer (useful for ML models):

```
1 df["adult_male"] = df["adult_male"].astype(int)
2 df["alone"]      = df["alone"].astype(int)
```

Verify the conversion:

```
1 df.dtypes
```

9 Text Cleaning

Inspect unique values:

```
1 df["sex"].unique()
2 df["class"].unique()
```

Standardize text values (lowercase, trim whitespace):

```
1 df["sex"]      = df["sex"].str.lower()
2 df["embark_town"] = df["embark_town"].str.strip()
```

Verify again:

```
1 df["sex"].unique()
```

10 Outlier Detection and Treatment

We use the **Interquartile Range (IQR)** method on the **fare** column.

10.1 Visualize Outliers

```
1 sns.boxplot(x=df["fare"])
2 plt.title("Fare Distribution")
3 plt.show()
```

10.2 Calculate IQR Boundaries

```
1 Q1 = df["fare"].quantile(0.25)
2 Q3 = df["fare"].quantile(0.75)
3 IQR = Q3 - Q1
4
5 lower = Q1 - 1.5 * IQR
6 upper = Q3 + 1.5 * IQR
```

10.3 Identify Outliers

```
1 outliers = df[(df["fare"] < lower) | (df["fare"] > upper)]
2 outliers
```

10.4 Count Outliers

```
1 len(outliers)
```

10.5 Remove Outliers

```
1 df = df[(df["fare"] >= lower) & (df["fare"] <= upper)]
```

10.6 Compare Shape Before and After

```
1 df.shape
```

Note on Outlier Handling in Machine Learning

In machine learning, we usually do **not repeatedly remove outliers until none remain** because:

- Removing outliers repeatedly can lead to a significant loss of data.
- Some extreme values may represent legitimate observations rather than errors.
- Excessive removal of observations can introduce bias into the dataset and reduce model generalization.

Common approaches for handling outliers include:

1. Leaving the outliers unchanged if they are meaningful.
2. Capping extreme values (Winsorization).
3. Applying a transformation such as a logarithmic transformation.

For the Titanic dataset, many data scientists prefer applying a log transformation to the `fare` variable:

```
df["fare"] = np.log1p(df["fare"])
```

The `fare` variable is highly right-skewed, and many of the large fare values correspond to genuine first-class ticket prices rather than data entry errors. Applying the logarithmic transformation reduces skewness, stabilizes variance, and makes the distribution more suitable for machine learning algorithms.

11 Feature Understanding

11.1 Target Variable

```
1 df["survived"].value_counts()
```

11.2 Passenger Class

```
1 df["pclass"].value_counts()
```

11.3 Gender Distribution

```
1 df["sex"].value_counts()
```

12 Final Data Validation

```
1 # 1. Missing values
2 df.isnull().sum()
3
4 # 2. Data types
5 df.dtypes
6
7 # 3. Shape
8 df.shape
9
10 # 4. Summary statistics
11 df.describe()
```

13 Saving the Cleaned Dataset

```
1 df.to_csv("titanic_cleaned.csv", index=False)
```

The cleaned dataset is now ready to be used for feature engineering and model building.

14 Reusable Data Quality Report Function

A reusable function that returns a quick quality summary of any dataset:

```
1 def data_quality_report(df):
2     report = {
3         "shape": df.shape,
4         "missing_values": df.isnull().sum().to_dict(),
5         "duplicate_count": int(df.duplicated().sum()),
6         "data_types": df.dtypes.astype(str).to_dict(),
7     }
8     return report
9
10 # Example usage
11 data_quality_report(df)
```

15 Practice Tasks

Task 1 — Cleaning Report

Create a cleaning report that contains:

- Initial shape and final shape
- Missing values removed
- Duplicate rows removed
- Columns dropped
- Outliers removed

Task 2 — Build the Function

Implement the `data_quality_report(df)` function shown above and test it on the original (uncleaned) Titanic dataset.

Task 3 — Independent Cleaning

Perform the entire cleaning workflow from scratch without referring to this document.

16 Mini Project — Exploratory Analysis

After cleaning the data, answer the following:

1. Which passenger class had the highest survival rate?
2. Did females survive more than males?
3. What is the average age of survivors?
4. Which embarkation town had the highest survival rate?
5. Create the following plots:
 - Histogram of `age`
 - Countplot of `survived`
 - Boxplot of `fare`
 - Heatmap of correlations

17 Summary

The complete data cleaning pipeline followed in this document:

1. Load the dataset and inspect its structure.
2. Generate descriptive statistics for numeric and categorical columns.
3. Analyze missing values and apply appropriate fill / drop strategies.
4. Remove duplicate records.
5. Fix data types for ML compatibility.
6. Standardize text values.
7. Detect and remove outliers using the IQR method.
8. Validate the final cleaned dataset.
9. Export the cleaned dataset to CSV.

A clean dataset is the foundation of every successful Machine Learning model.